NAME : GURLEEN
COURSE : B.TECH CS (DS)
ROLL NO. : 2301420051

Capstone Assignment

COMPUTER ORGANISATION AND ARCHITECTURE

* Case Study: Performance Improvement of a College ERP Server:

- INTRODUCTION:

A college ERP (Enterprise Resource Planning) system is used for managing admissions, results, attendance and student data. During peak periods, such as admissions and result processing, the system becomes slow due to high user load and increased data processing. The aim is to improve system performance without replacing the entire infrastructure, focusing on processor and memory-level optimizations.

→ SOLUTION:

- AMDAHL'S LAW:

Amdahl's law helps determine the maximum improvement possible when only part of a system is enhanced.

$$\rightarrow \text{Speedup} = \frac{1}{(1-P) + \frac{P}{S}}$$

where,

P : fraction of system improved

S : speedup of improved part

- INSTRUCTION PIPELINING:

FETCH → DECODE → EXECUTE → MEMORY → WRITE BACK

This allows multiple instructions to overlap, increasing throughput.

- CACHE MEMORY:

It is small, fast memory between CPU and RAM:
Reduces memory access time.
Improves execution speed.
Types: L1, L2, L3 cache.

- **Performance Bottlenecks :**

CPU speed limitations, memory access delay, poor cache utilization, pipeline hazards.

- **Proposed Analysis :**

Amdahl's Law Application :

Assume ERP workload distribution :

70% time → Memory operations (database access, fetching records)

30% time → CPU processing (calculations, logic execution)

Example :

Suppose,

- CPU improved by 2x.
- Memory improved by 2x.

$$\text{Speedup} : \frac{1}{\frac{0.7}{2} + \frac{0.3}{2}} = \frac{1}{0.5} = 2$$

∴ 100% improvement (2x faster system)

- **Memory Bottleneck :**
  - Data stored in large databases.
  - Every request : multiple memory accesses.
  - RAM slower than CPU.
  - CPU often waits idle called Memory Stall

$$\text{CPU utilization} = \frac{\text{Execution time}}{\text{Execution + Stall Time}}$$

- **Cache Impact Analysis :**

Suppose :

Cache Hit Rate = 90%

Miss Penalty = High

Effective Access Time = Hit Time + Miss Rate × Miss Penalty

Even small cache improvements turns big system speedup.

- **Pipeline Performance in ERP:**

Multiple instructions overlap and Throughput increases.

$$\text{Throughput} : \frac{\text{No. of instructions}}{\text{Time}}$$

...ine Issues :

a

In pipelined processor, performance improves because multiple instructions executed simultaneously. However, this efficiency is often reduced by pipeline hazards which introduce delays (stalls) and prevent the pipeline from achieving ideal speedup.

1. Data Hazards :

Occurs when one instruction depends on the result of a result of a previous instructions.

Example :

I_1 : $A = B + C$

I_2 : $D = A + E$

→ Solution : Data forwarding and pipeline stalling.

2. Control Hazards :

Occurs due to branch instructions

Example :

if (marks > 50)

→ Solution : Branch Prediction & Speculative Execution.

3. Structural Hazards :

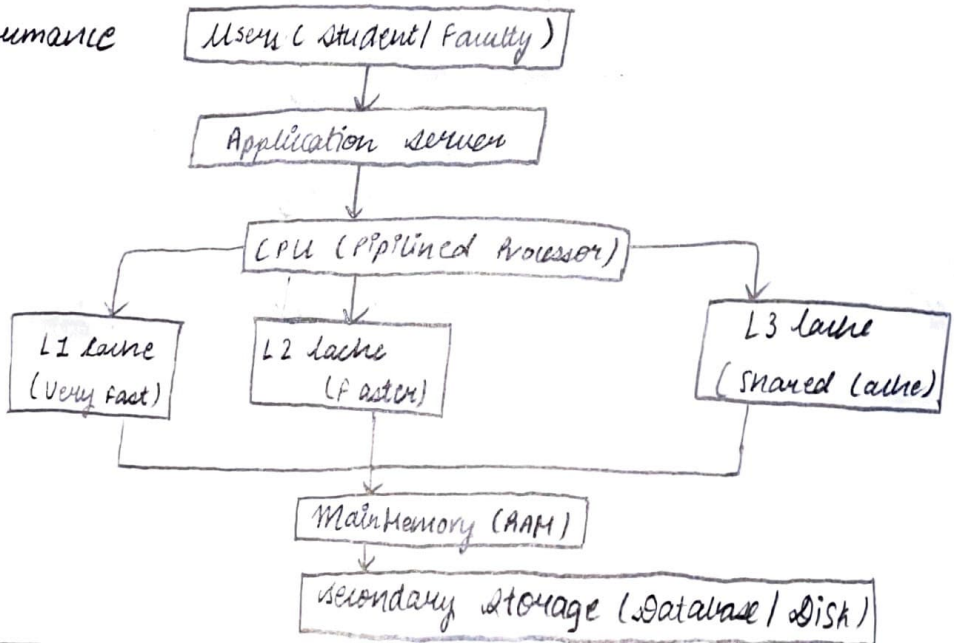
Occurs when hardware resources are limited.

→ Solution : Separate instruction & data memory.

4. Memory stall Solution :

→ • Use cache memory
  • Prefetching
  • Faster RAM

Ent Server Performance Improvement



- Trade-off Analysis:

The trade-off analysis highlights that no single hardware improvement can completely solve the EPP performance issue, as each component has its own advantages and limitations. Upgrading the processor increases computational speed, but its impact is limited because the system spends a significant amount of time waiting for memory access. Increasing cache memory is highly effective in reducing memory latency and improving access time, but it comes with higher cost & design complexity. Instruction Pipelining improves throughput by allowing multiple instructions to execute simultaneously. However, it's efficiency is affected by hazards such as data and control dependencies which may introduces delays. Therefore, the best approach is a balanced optimization strategy that combines cache enhancement, efficient pipelining and moderate processor and memory upgrades to achieve maximum performance improvement in a cost-effective manner.

→ Pipelining (Instruction):

	1	2	3	4	5	6	7
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	EX	MEM	WB	
I_3			IF	ID	EX	MEM	WB

∴ Therefore, overlapping execution increases throughput.

→ Cache Memory

CPU → Cache → Main Memory → Disk

↓

Hit → Data Returned

Miss → Access Ram → Return Data

Effective Access Time = Hit Time + (Miss Rate × Miss Penalty).

Recommendations:

Based on the overall analysis of the ERP system using concepts such as Amdahl's law, pipelining and cache memory, it is clear that the system's performance is mainly limited by memory access delays rather than pure CPU computation.

Key Recommendations:

1. Cache Memory optimization:
   - Increase cache size.
   - Use set-associative mapping to reduce misses.
   - Store frequently accessed ERP data.
2. Efficient Instruction Pipelining:
   - Enabling smooth execution of multiple instructions.
   - Apply techniques like data forwarding and branch prediction.
3. Memory Performance Improvement:
   - Upgrade to faster RAM (low-latency).
   - Use prefetching techniques, reduces memory stalls.
4. Software-Level optimization:
   - Optimize database queries (indexing, caching).
   - Reduce redundant data access.
5. Balanced System Design:
   - Combine CPU, cache and memory improvements.
   - It avoids limitation prediction by Amdahl's law.

- Therefore, a balanced approach that integrates hardware-level improvements with software optimizations ensures maximum performance gain, cost efficiency and scalability, making it the best recommendation for handling high-load scenarios in a college ERP system.

Case Study 4 : Cache Design for an Online Examination System.

* INTRODUCTION :

An online examination system is accessed by thousands of students simultaneously, especially during peak exam time.

Problems addressing :

- Slow response time
- Delay in loading questions
- High server load

Design an efficient memory hierarchy strategy to improve system performance using LOA :

→ Concepts :

- Locality of Reference :
  Programs tend to access the same data repeatedly.

→ Types :

1. Temporal Locality : Recently accessed data is reused.
   Example - Same exam questions accessed multiple times.
2. Spatial Locality : Nearby data is accessed.
   Example - Loading next question

CPU → Cache → Main Memory → Disk

- Cache Memory :
  Cache is a small, high-speed memory placed between CPU and RAM.

Cache Mapping Technique :

- Direct Mapping :
  Each memory block → only one cache location.

  Memory Block → Fixed Cache Line

- Fully Associate Mapping :
  Any block → any cache location.

  Memory Block → Any Cache Line

Cache Block

Associative Mapping:
divided into sets.
maps to a set, not fixed line.

Memory → set → any line in set.

* Trade-off in Cache Design:

Cache design is an online examination system involves a careful balance between performance, cost and complexity. Increasing the cache size improves the hit rate and reduces memory access time, but it also increases hardware cost and may slightly increase access latency. Similarly, a larger block size helps in exploiting spatial locality, allowing more nearby data to be fetched at once. However, it increases the miss penalty and may waste bandwidth if unnecessary data is loaded. Increasing associativity reduces conflict misses and improves cache efficiency, but it makes cache lookup slower and the design more complex.

→ Performance Issues:

1. **Cache Misses**: When required data is not found in cache, the system must access main memory, causing delay.
   Example: First time loading a question → miss → slower response.
2. **High Miss Penalty**: When a cache miss occurs, fetching data from RAM takes much more time.
   Example: If exam questions are not cached, each request goes to main memory → delay increases.
3. **Memory Stall**: CPU has to wait when data is not available immediately.
4. **Poor Locality of Reference**: If the program does not reuse data efficiently, cache hit rate decreases and performance drops.
5. **Conflict Misses**: In direct mapping, multiple memory blocks map to the same cache line.

★ Recommendations:

→ To achieve high performance in an online examination system with heavy concurrent users, an optimized cache design must be implemented using COA principles. The system should adopt a multi-level cache hierarchy (L1, L2, L3) to minize memory access time and reduce CPU waiting. Among mapping techniques, set associative mapping is recommended as it provides a balanced trade-off between speed, cost and miss rate.

Further, a write-back cache policy with write-allocate should be used to improve performance by reducing unnecessary memory writes. Also, maintaining a high cache hit rate is essential to ensure low Average Memory Access Time. Overall, combining efficient cache hierarchy, optimized mapping and proper write policy will significantly improve system responsiveness, reduce latency and ensure smooth execution of the online examination system even under heavy load.